

Needleman-Wunsch Global Sequence Alignment Tool - Python

Weronika Łaszkiewicz, 278434

30.04.2025

1 Introduction

The Needleman-Wunsch algorithm represents a fundamental method in bioinformatics for performing global sequence alignment. This project implements a complete Python solution that combines the classical dynamic programming approach with modern visualization capabilities. The tool enables quantitative comparison of both DNA and protein sequences while providing intuitive graphical representation of the alignment process.

The complete source code and documentation are available in the project's GitHub repository:

github.com/wlaszkiewicz/needleman-wunsch

2 System Design

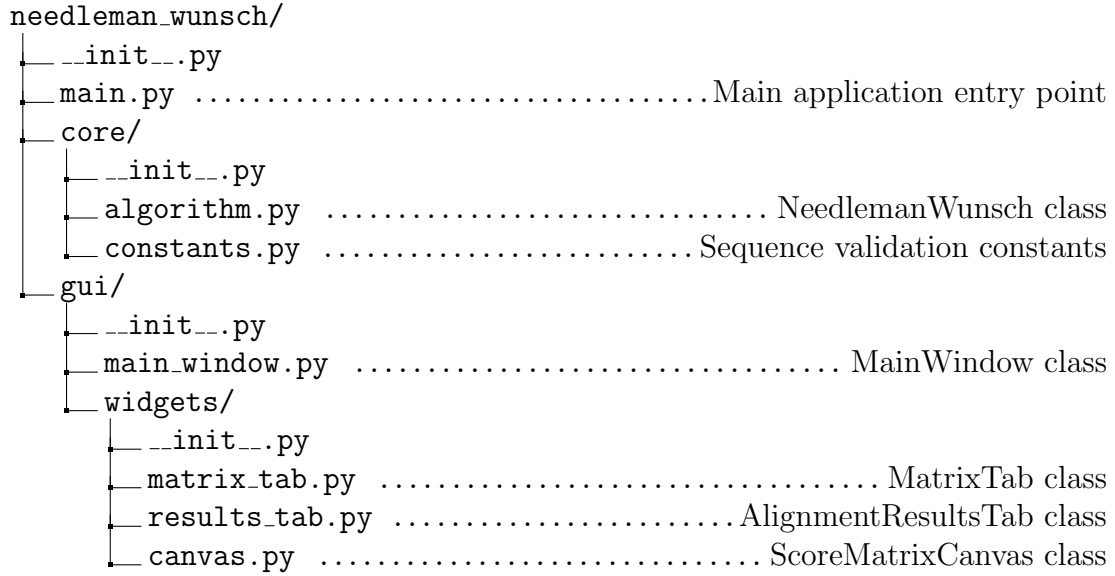
2.1 Project Structure

The implementation follows a modular design organized into three main components:

- **Core Algorithm Module:**
 - `algorithm.py`: Contains the NeedlemanWunsch class implementing the dynamic programming solution
 - `constants.py`: Stores biological constraints for sequence validation
- **GUI Module:**
 - `main_window.py`: Main application window and controller logic
 - `widgets/matrix_tab.py`: Score matrix visualization component
 - `widgets/results_tab.py`: Alignment results display
 - `widgets/canvas.py`: Interactive score matrix canvas
- **Application Entry:**
 - `main.py`: Launches the Qt application and main window

This modular architecture separates concerns between the algorithmic core and presentation layer, enabling easier maintenance.

2.2 Structure



3 Algorithm Implementation Details

3.1 Mathematical Foundation

The Needleman-Wunsch algorithm solves the global alignment problem through dynamic programming by building a score matrix S where each cell $S[i][j]$ represents the optimal score for aligning the first i characters of sequence A with the first j characters of sequence B . The recurrence relation is:

$$S[i][j] = \max \begin{cases} S[i-1][j-1] + \text{match/mismatch score} \\ S[i-1][j] + \text{gap penalty} \\ S[i][j-1] + \text{gap penalty} \end{cases} \quad (1)$$

3.2 Algorithm Overview

The implementation follows the standard Needleman-Wunsch approach with three main phases:

1. **Initialization:** Constructs the scoring matrix with gap penalties
2. **Matrix Filling:** Computes alignment scores using dynamic programming
3. **Traceback:** Determines the optimal alignment path(s)

3.3 Core Algorithm

The key components are implemented in the `NeedlemanWunsch` class:

Algorithm 1 Needleman-Wunsch Alignment

```
1: Initialize scoring matrix with gap penalties
2: for  $i \leftarrow 1$  to  $m$  do
3:   for  $j \leftarrow 1$  to  $n$  do
4:      $match \leftarrow S[i-1][j-1] + \delta(a_i, b_j)$ 
5:      $delete \leftarrow S[i-1][j] + gap$ 
6:      $insert \leftarrow S[i][j-1] + gap$ 
7:      $S[i][j] \leftarrow \max(match, delete, insert)$ 
8:   end for
9: end for
10: Traceback from  $S[m][n]$  to find optimal path(s)
```

3.4 Key Features

- Sequence validation for DNA/protein inputs
- Configurable scoring parameters:
 - Match score (default: +1)
 - Mismatch penalty (default: 0)
 - Gap penalty (default: -1)
- Multiple optimal path detection (up to 10 paths)
- Alignment statistics:
 - Identity percentage
 - Gap percentage
 - Alignment score

3.5 Key Methods

3.5.1 Initialization

The algorithm begins by setting up the scoring parameters and empty data structures:

```
1 def __init__(self, match_score=1, mismatch_penalty=0, gap_penalty=-1):
2     self.match_score = match_score
3     self.mismatch_penalty = mismatch_penalty
4     self.gap_penalty = gap_penalty
5     self.seq1 = ""
6     self.seq2 = ""
7     self.seq_type = None
8     self.score_matrix = []
9     self.alignment_paths = []
10    self.total_optimal_paths = 0
11    self.alignment_examples = []
12    self.alignment_score = 0
```

3.5.2 Sequence Validation

The input sequences are validated according to biological constraints:

```
1 def detect_sequence_type(self):
2     is_dna = all(c in DNA_BASES for c in self.seq1 + self.seq2)
3     is_protein = all(c in PROTEIN_RESIDUES for c in self.seq1 + self.seq2)
4
5     if is_dna: return 'dna'
6     if is_protein: return 'protein'
7     raise ValueError("Sequences contain invalid characters")
```

3.6 Dynamic Programming Matrix Construction

The core alignment process consists of:

Matrix Initialization

```
1 self.score_matrix = np.zeros((len1 + 1, len2 + 1), dtype=int)
2 for i in range(len1 + 1):
3     self.score_matrix[i][0] = i * self.gap_penalty
4 for j in range(len2 + 1):
5     self.score_matrix[0][j] = j * self.gap_penalty
```

Matrix Filling

```
1 for i in range(1, len1 + 1):
2     for j in range(1, len2 + 1):
3         match = self.score_matrix[i-1][j-1] + (
4             self.match_score if self.seq1[i-1] == self.seq2[j-1]
5             else self.mismatch_penalty)
6         delete = self.score_matrix[i-1][j] + self.gap_penalty
7         insert = self.score_matrix[i][j-1] + self.gap_penalty
8         self.score_matrix[i][j] = max(match, delete, insert)
```

3.7 Path Tracing

The traceback algorithm handles both single and multiple optimal paths. It recursively finds all optimal alignment paths through the score matrix. The number of optimal paths can grow exponentially with sequence length. To maintain reasonable performance, we limit the search to max_paths to 10.

Algorithm 2 Multiple Path Traceback

```
1: procedure FIND_ALL_OPTIMAL_PATHS(i, j, path, paths, max_paths)
2:   if path count  $\geq$  max_paths then
3:     return
4:   end if
5:   Add current cell to path
6:   if reached origin (i=0 and j=0) then
7:     Store reversed path
8:     return
9:   end if
10:  for each valid move (diagonal, up, left) do
11:    if score transition is valid then
12:      Recursively continue path
13:    end if
14:  end for
15: end procedure
```

Code sample

```
1      def find_all_optimal_paths(self, i, j, path=None, paths=None,
2      ↪ max_paths=10):
3
4          if paths is None:
5              paths = []
6          if path is None:
7              path = []
8
9          # Early termination to prevent combinatorial explosion
10         if len(paths) >= max_paths:
11             return
12
13         path.append((i, j))
14
15         # Base case when complete path found (origin reached):
16         if i == 0 and j == 0:
17             paths.append(path[::-1])
18             self.total_optimal_paths = len(paths)
19             return
20
21         # Recursive case:
22         current_score = self.score_matrix[i][j]
23         possible_moves = []
24
25         # 1. Diagonal move (match/mismatch)
26         if i > 0 and j > 0:
27             diagonal_score = self.score_matrix[i - 1][j - 1]
28             match_value = self.match_score if self.seq1[i - 1] ==
29             ↪ self.seq2[j - 1] else self.mismatch_penalty
30             if current_score == diagonal_score + match_value:
31                 possible_moves.append((i - 1, j - 1))
32
33         # 2. Vertical move (gap in seq2)
```

```

32     if i > 0 and current_score == self.score_matrix[i - 1][j] +
    ↪ self.gap_penalty:
33         possible_moves.append((i - 1, j))
34
35     # 3. Horizontal move (gap in seq1)
36     if j > 0 and current_score == self.score_matrix[i][j - 1] +
    ↪ self.gap_penalty:
37         possible_moves.append((i, j - 1))
38
39     for move in possible_moves:
40         if len(paths) >= max_paths:
41             break
42         self.find_all_optimal_paths(move[0], move[1], path.copy(),
    ↪ paths, max_paths)
43
44     self.alignment_paths = paths
45     self.total_optimal_paths = len(paths)

```

3.8 Complexity Analysis

The implementation demonstrates the following theoretical complexities:

Operation	Time	Space
Matrix Construction	$O(mn)$	$O(mn)$
Single Path Traceback	$O(m + n)$	$O(m + n)$
Multiple Paths (k paths)	$O(k(m + n))$	$O(k(m + n))$

Table 1: Computational complexity by operation

Where:

- m = length of sequence 1
- n = length of sequence 2
- k = number of optimal paths (limited to 10)

4 User Interface

4.1 GUI Components

The PyQt5 interface (Figure 1 and 2) provides:

- Sequence input fields (manual or FASTA import)
- Parameter configuration panel
- Results display with alignment visualization
- Score matrix visualization with path tracing
- Export functionality for results and graphics

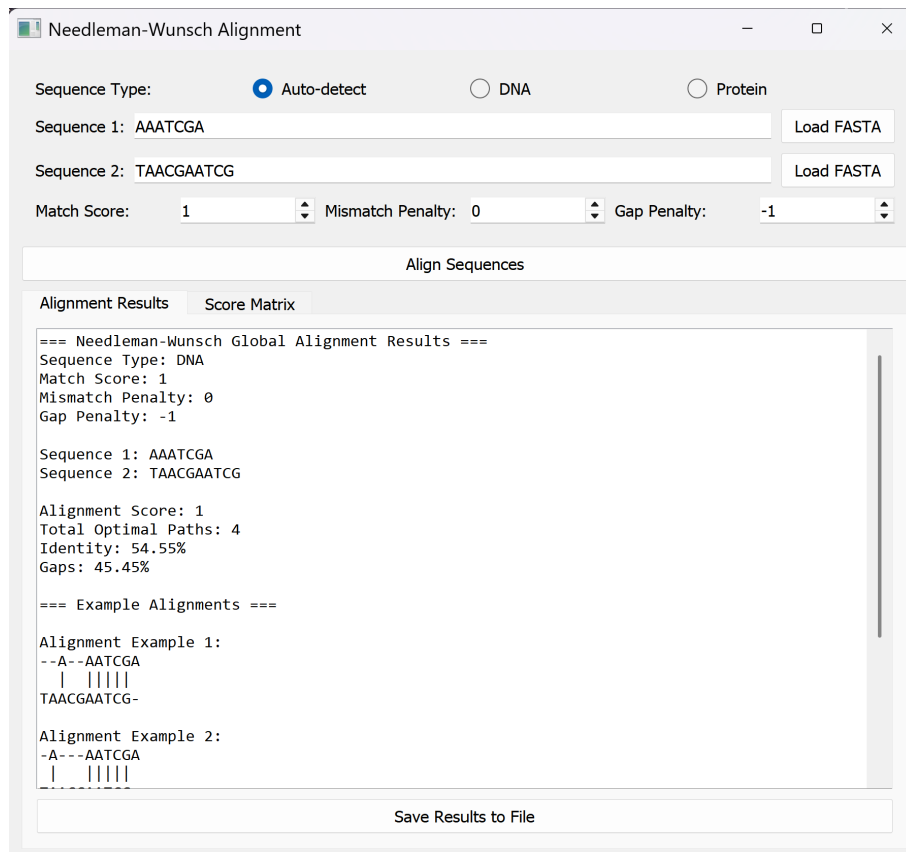


Figure 1: Program graphical user interface

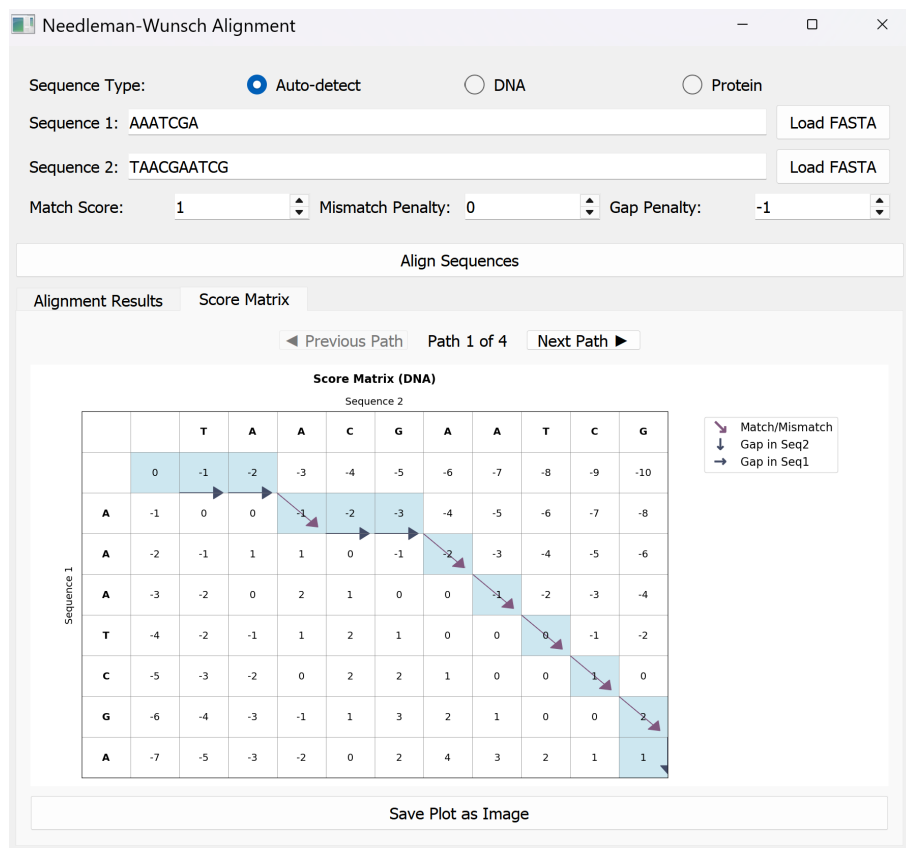


Figure 2: Program graphical user interface

5 User Manual

5.1 Basic Operation

1. Select sequence type (Auto-detect/DNA/Protein)
2. Input sequences manually or load FASTA files
3. Adjust scoring parameters if needed
4. Click "Align Sequences"
5. View results in the tabs:
 - Alignment Results: Textual alignment
 - Score Matrix: Visual path representation
6. Use navigation buttons to view alternate optimal paths (up to 10)
7. Save results via the export buttons

5.2 Input Requirements

- **DNA:** Only A,T,C,G,U,N characters allowed
- **Protein:** Standard 20 amino acid codes plus special characters
- **FASTA:** Standard format with optional header lines

6 Results Interpretation

The program generates:

- Optimal alignment(s) with match indicators
- Alignment statistics (score, identity, gaps)
- Visual representation of the scoring matrix
- Path arrows indicating alignment moves:
 - Diagonal: Match/mismatch
 - Vertical: Gap in sequence 2
 - Horizontal: Gap in sequence 1

7 Example Results

7.1 Alignment Results

```
1 === Needleman-Wunsch Global Alignment Results ===
2 Sequence Type: PROTEIN
3 Match Score: 1
4 Mismatch Penalty: 0
5 Gap Penalty: -1
6
7 Sequence 1: CATWALK
8 Sequence 2: CQWARD
9
10 Alignment Score: 2
11 Total Optimal Paths: 2
12 Identity: 42.86%
13 Gaps: 14.29%
14
15 === Example Alignments ===
16
17 Alignment Example 1:
18 CATWALK
19 | *||**
20 C-QWARD
21
22 Alignment Example 2:
23 CATWALK
24 |* ||**
25 CQ-WARD
```

7.2 Score Matrix

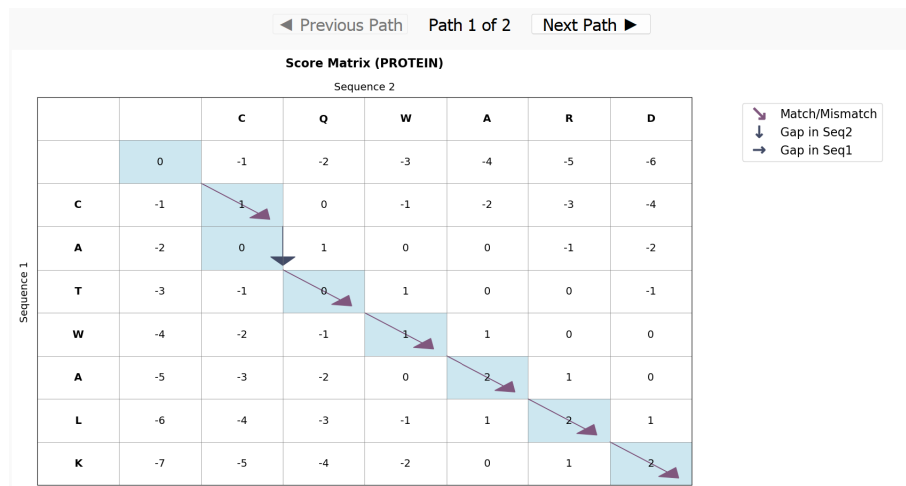


Figure 3: Exemplary Matrix Result - Path 1

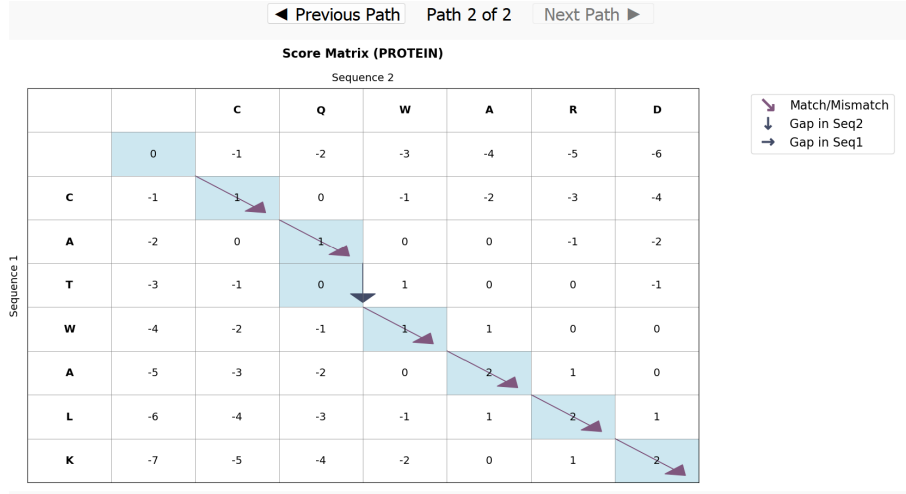


Figure 4: Exemplary Matrix Result- Path 2

8 Conclusion

The implemented Needleman-Wunsch alignment tool successfully meets all specified requirements, providing a complete solution for global sequence alignment.

The system supports both manual sequence entry and FASTA file loading, with automatic DNA/protein detection and input validation. Users can customize all scoring parameters (match, mismatch, and gap penalties) through an intuitive graphical interface. The program generates optimal alignments with comprehensive statistics including alignment scores, identity percentages, and gap analysis.

A key feature is the ability to find and visualize all optimal alignment paths (up to 10) with interactive navigation between different solutions. The graphical display clearly shows the scoring matrix with highlighted paths, allowing users to understand the alignment process through directional arrows for matches, mismatches, and gaps. While the multiple path generation has practical limits for computational efficiency, the implementation provides valuable insight into alternative optimal alignments.